

2.6 INTER-PROCESS COMMUNICATION

Cooperating processes require some mechanism to exchange data or pass information to each other. One such mechanism is **inter-process communication (IPC)**—a very useful facility provided by the operating system. IPC allows the processes running on a single system to **communicate with each other**. Two basic communication models for providing IPC are **shared memory systems** and **message passing systems**. In the former model, a part of memory is shared among the cooperating processes. The processes that need to exchange data or information can do so by writing to and reading from this shared memory. However, in the latter model, the cooperating processes communicate by sending and receiving messages from each other. The communication using message passing is much more time consuming as compared to shared memory. This is because the message passing system is implemented with the help of operating system calls and thus, it requires a major involvement of kernel. On the other hand, in shared memory systems, system calls are used only to set up the shared memory area. Once the shared area is set up, no further kernel intervention is required.

2.6.1 Shared Memory Systems

In shared memory systems, **the process that needs to communicate with the other processes creates a shared memory segment in its own address space**. Other processes can communicate with this process by **attaching its shared memory segment along with their address space**. All the communicating processes can read or write data through this shared area. Note that these processes must be synchronized so that no two processes are able to access the shared area simultaneously. [Figure 2.6](#) shows a shared memory communication model.

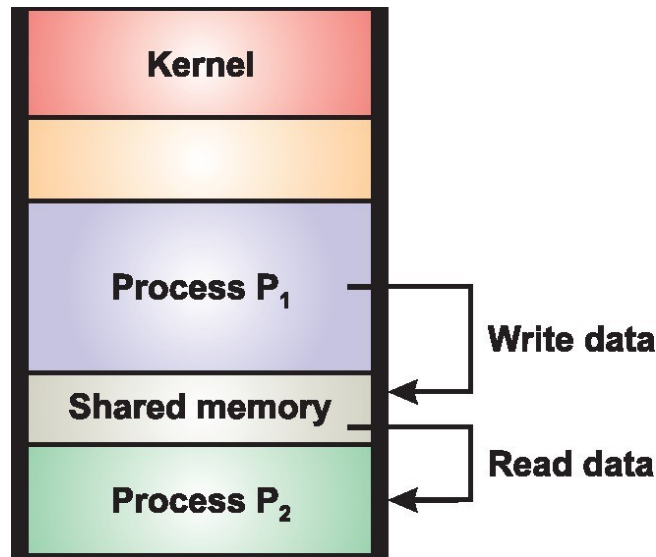


Fig. 2.6 Shared Memory Communication Model

To understand the concept of shared memory systems, consider a common example of cooperating processes known as **producer-consumer** problem. In this problem, there are two processes, one is *producer* that produces the items and the other is *consumer* that consumes the items produced by the producer. These two processes need to run concurrently thereby requiring communication with each other. One possible solution to this problem can be provided through shared memory. Both the producer and consumer processes are made to share a common **buffer** between them. The producer process fills the buffer by placing the produced items in it and the consumer process vacates the buffer by consuming these items.

The buffer shared between producer and consumer processes may be bounded or unbounded. In bounded buffer, the size of buffer is fixed; therefore, the producer process has to wait in case the buffer is full; similarly the consumer process has to wait in case the buffer is empty. On the other hand, in unbounded buffer, there is no limit on the buffer size. Thus, only the consumer process has to wait in case there is no item to be consumed. However, the producer process need not wait and it may continuously produce items.

2.6.2 Message Passing Systems

In message passing systems, two system calls, `send()` and `receive()`, are used.

The sender process (say, P_1) sends the message to the operating system by invoking the `send()` system call.

The operating system stores this message in the buffer area until the `receive()` system call is invoked by the receiver process (say, P_2).

After that the operating system delivers this message to P_2 .

In case there is no message available for P_2 when it invokes the `receive()` system call, the operating system blocks it until some message arrives for it.

On the other hand, if a number of messages arrive for P_2 , the operating system puts them in a queue and delivers them in FIFO order upon the invocation of `receive()` call (one for each process) by P_2 .

Figure 2.7 shows the message passing communication model.

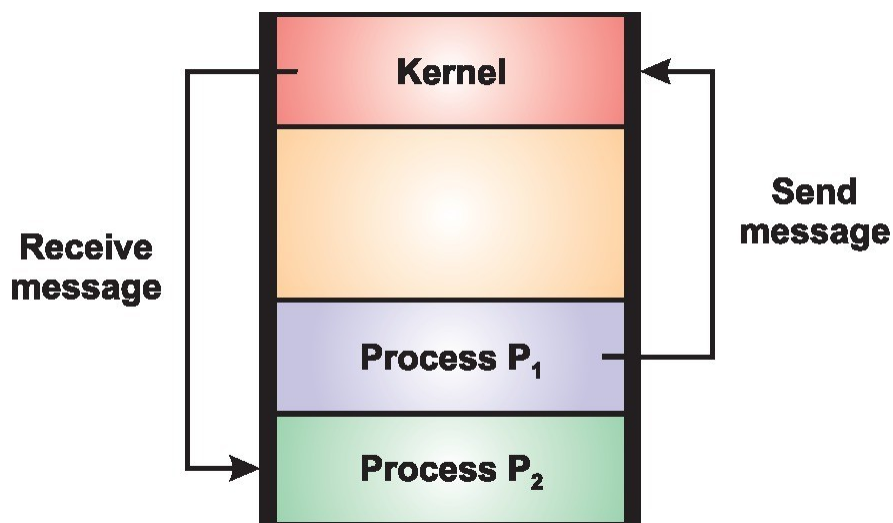


Fig. 2.7 Message Passing Communication Model

In message passing, it is not necessary for the communicating processes to reside on the same computer rather they may reside on different computers connected via a network (a distributed environment). Therefore, whenever two processes want to communicate, a

communication link must be established between them.

At the physical level, the communication link may be implemented via shared variables or bus or the network, etc. However, at the logical level, some features related with the implementation of communication link arise.

Types of Communication

Processes may communicate with each other directly or indirectly.

In **direct communication**, processes address each other by their `PID` assigned to them by the operating system. For example, if a process P_1 wants to send a message to process P_2 , then the system calls `send()` and `receive()` will be defined as follows:

- `send(PID2, message)`
- `receive(PID1, message)`

Since both sender and receiver process need to know each other's `PID`, this type of communication is known as symmetric direct communication. However, **asymmetry in addressing** can be represented by making only the sender process to address the receiver process by its `PID` but the receiver process need not know the `PID` of the sender process. In the case of asymmetric direct communication, the calls `send()` and `receive()` will be defined as follows:

- `send(PID2, message)`
- `receive(id, message)`

Now, when the operating system delivers a message to process P_2 upon the invocation of a `receive()` call by it, the parameter `id` is replaced with the `PID` of the sender process.

In **indirect communication**, messages are sent and received via **mailbox** (also known as **port**)—a repository of inter-process messages. A mailbox, as the name implies, is just like a postbox into which messages sent by the processes can be stored and removed by other processes. The different characteristics of a mailbox are as follows:

- Each mailbox has a unique ID and the processes communicate with each other through a number of mailboxes.
- The process that creates the mailbox is the owner of mailbox and

only this process can receive messages from it. Other processes can only send messages to it. In other words, there can be multiple senders but a single recipient for a mailbox.

- The process that knows the ID of a mailbox can send messages to it.
- Besides a user process, the operating system may also own a mailbox. In this case, the operating system may allow the processes to create or delete a mailbox, send and receive messages via mailbox. The process that creates the mailbox becomes the owner of that mailbox and may receive messages through this mailbox. However, with time, other processes can also be made to receive messages through this mailbox by passing ownership to them.

The system calls to send a message to a mailbox (say, x) and receive a message from a mailbox will be defined as follows:

- `send( $X$ , message)`
- `receive( $X$ , message)`

As stated earlier, a communication link must exist between processes before starting the communication. The communication link exhibits different properties in direct and indirect communication, which are discussed in [Table 2.1](#).

Table 2.1 Comparison of Direct and Indirect Communication

<i>Direct communication</i>	<i>Indirect communication</i>
• There exists only one link between each pair of communicating processes.	• There may be multiple links between each pair of communicating processes, where each link corresponds to exactly one mailbox.
• A link is associated with just two processes.	• A link may be associated with more than two processes.
• The link is established automatically between the communicating processes, provided the sender process knows the PID of	• The communication link can be established between two processes only if both the communicating processes share a mailbox with

the receiver process.	each other.
-----------------------	-------------

Synchronization

Messages can be sent or received either **synchronously** or **asynchronously**, also called **blocking** or **non-blocking**, respectively. Various design options for implementing `send()` and `receive()` calls are as follows:

- **Blocking send:** If a process (say, P_1) invokes `send()` call to send a message to another process (say, P_2) or to a mailbox, the operating system blocks P_1 until the message is received by P_2 or by the mailbox.
- **Blocking receive:** If there is no message available for P_2 when it invokes the `receive()` system call, the operating system blocks it until some message arrives for it.
- **Non-blocking send:** P_1 sends the message and continues to perform its operation without waiting for the message delivery by P_2 or by mailbox.
- **Non-blocking receive:** When P_2 invokes a `receive()` call, it either gets a valid message if some message is available for it or `NULL` if there is no message available for it.

Buffering

As discussed earlier, the messages sent by a process are temporarily stored in a temporary queue (also called **buffer**) by the operating system before delivering them to the recipient. This buffer can be implemented in a variety of ways, which are as follows:

- **No buffering:** The capacity of buffer is zero, that is, no messages may wait in the queue. This implies that sender process has to wait until the message is received by the receiver process.
- **Bounded buffer:** The capacity of the buffer is fixed, say m , that is, at most m processes may wait in the queue at a time. When there are less than m messages waiting in the queue and a new message arrives, it is added in the queue. The sender process need not wait

and it can resume its operation. However, if the queue is full, the sender process is blocked until some space becomes available in the queue.

- **Unbounded buffer:** The buffer has an unlimited capacity, that is, an infinite number of messages can be stored in the queue. In this case, the sender process never gets blocked.
- **Double buffering:** Two buffers are shared between the sender and receiver process. In case one buffer fills up, the second one is used. When the second buffer fills up, the first might have been emptied. This way the buffers are used turn by turn, thus avoiding the blocking of one process because of another.

Pipes

Pipe is the standard communication mechanism that enables the transfer of data between processes. It provides a means of one-way communication between related processes. Each pipe has a read end and a write end. The data written at the write end of the pipe can be seen (read) through the read end. When the writer process writes to the pipe, stream of bytes are copied to the shared buffer whereas at the time of reading bytes are copied from the shared buffer.

Though both the reader and writer processes may run concurrently, the access to pipe must be synchronized. UNIX must ensure that only one process (either writer or reader) is accessing the pipe at a time. To synchronize the processes, UNIX uses locks and wait queues; each end of pipe is associated with a wait queue. Whenever the writer process requests for writing to pipe, UNIX locks the pipe for it if and only if there is enough space as well as the pipe is not locked for the reader process. Once the writer process gains access to pipe, bytes are copied into it. However, if the pipe is full or locked for the reader process, the writer process sleeps in the wait queue at write end and remains there unless it is not awakened by the reader. After the data has been written to the pipe, the pipe is unlocked and any sleeping readers in the wait queue at read end are awakened. A similar process follows at the time of reading from the pipe.

Here are some advantages and disadvantages of using pipes as an IPC technique:

Advantages:

Simplicity: Pipes are a **simple and straightforward way** for processes to communicate with each other.

Efficiency: Pipes are an **efficient way for processes to communicate**, as they can **transfer data quickly** and with minimal overhead.

Reliability: Pipes are a reliable way for processes to communicate, as they **can detect errors in data transmission and ensure that data is delivered correctly**.

Flexibility: Pipes **can be used to implement various communication protocols**, including one-way and two-way communication.

Disadvantages:

Limited capacity: Pipes have a limited capacity, **which can limit the amount of data that can be transferred** between processes at once.

Unidirectional: In a unidirectional pipe, only **one process can send data at a time**, which can be a disadvantage in some situations.

Synchronization: In a bidirectional pipe, **processes must be synchronized** to ensure that data is transmitted in the correct order.

Limited scalability: Pipes are limited to communication between a **small number of processes on the same computer**, which can be a disadvantage in large-scale distributed systems.

Overall, pipes are a useful IPC technique for simple and efficient communication between processes on the same computer. However, they may not be suitable for large-scale distributed systems or situations where bidirectional communication is required.

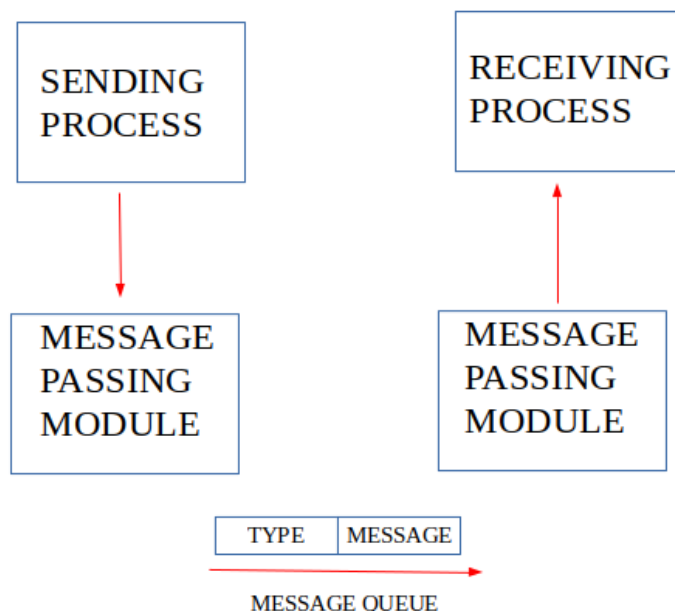
IPC using Message Queue

A Message Queue is a linked list of messages stored within the kernel and identified by a message queue identifier and it is used to send and receive messages between processes in a **first-in-first-out (FIFO)** manner.

The **msgget()** function is used to create a new queue or open an existing queue and new messages are added to the end of a queue by using **msgsnd()**.

In a message queue every message has a positive long integer type field, a non-negative length, and the actual data bytes (corresponding to the length), all of which are specified to **msgsnd()** when the message is added to a queue. Messages are fetched from a queue by **msgrcv()**.

Note: We don't have to fetch the messages in a first-in, first-out order. Instead, we can fetch messages based on their type field.



IPC Using Message Queues

The Inter-Process Communication (IPC) is essential for the enabling processes to the communicate and synchronize with the each other in an operating system. Among the various IPC mechanisms message queues provide the robust and flexible method for the processes to

exchange data in a queued orderly fashion. It enables asynchronous communication, where messages are placed in a queue and retrieved by processes in a convenient manner.

Working of IPC using Message Queue

Creating/Accessing a Message Queue: The first step is to create or access a message queue using the **msgget()** function. The msgget function creates a new message queue if the queue does not exist.

Sending the Message: After successfully creating/accessing the message queue a process can now send a message to the queue using **msgsnd()** function which will be stored in the message queue in a format that contains message type and actual message content.

Receiving the Message: Once the message is added in the queue, it can be retrieved from the queue by another process using **msgrcv()**. This function allows the receiving process to specify the message type it wants to read. The message is then removed from the queue once it is received.

Delete the Message Queue: We can use the **msgctl()** function to delete the message queue if it is no longer needed.

Functions of Message Queue

The Message queues are powerful IPC tools that allow processes to send and receive messages in the FIFO (First In, First Out) order. Here are the primary functions of the message queues:

Message Storage: A message queue stores messages sent by the one process until they are retrieved by the another process. Each message is placed in the queue and remains there until it is read by the receiving process.

Ordered Communication: The Message queues ensure that messages are delivered in the order they were sent. This ordered communication is vital in the scenarios where the sequence of the operations matters.

Asynchronous Communication: With message queues, processes do not need to be synchronized or directly connected. A sending process can place a message in the queue and receiving process can retrieve it later allowing for the asynchronous communication.

Decoupling of Processes: The Message queues allow processes to be decoupled meaning the sending and receiving processes do not need to be aware of the each other's existence or state. They interact indirectly through the queue which can improve modularity and scalability.

Prioritization: The Some message queue implementations allow the messages to be prioritized where certain messages can be processed before others based on the priority levels.

Error Handling: The Message queues can be designed to handle errors such as retries for the failed message deliveries or logging of the undelivered messages for the troubleshooting.

System calls used for message queues:

ftok(): to generate a unique key.

msgget(): to Create a new queue or access an existing queue either returns the message queue identifier for a newly created message queue or returns the identifiers for a queue which exists with the same key value.

msgsnd(): to add message to a message queue.

msgrcv(): for retrieving messages from a queue.

msgctl(): used to delete message queue.